RiderGuy Build Plan

Phased Sprint-by-Sprint Development Roadmap

From Zero to a Fully Operational 4-in-1 Platform

RiderGuy Global

February 2026

RiderGuy — Build Plan

> *A phased, sprint-by-sprint plan to take RiderGuy from zero to a fully operational 4-in-1 platform. Each phase builds on the last. Every sprint delivers working, testable software.*

Build Philosophy

1. **Ship early, ship often** — get a working MVP into real riders' hands as fast as possible
2. **Backend-first** — the API powers all four platforms; build it once, consume it four times
3. **PWA-native** — every frontend is a Progressive Web App from day one (installable, offline-capable, responsive)
4. **Monorepo** — all four frontends + the shared backend live in one repository for consistency and speed
5. **Feature flags** — every major feature ships behind a toggle; enable per region, per user segment, or globally
6. **Mobile-first design** — the rider app and client app are designed for phones first, then scale up
7. **Offline-first where it counts** — riders in the field can't afford to lose connectivity; critical flows work offline
8. **Test as you build** — automated tests from sprint 1; no "we'll add tests later"

Technical Stack (Decided)

Layer	Technology	Why
Frontend Framework	Next.js 14+ (React)	SSR for marketing site SEO, app router for PWA shells, massive ecosystem, PWA support via next-pwa
Language	TypeScript (full stack)	Type safety across frontend and backend, shared types, fewer bugs
Styling	Tailwind CSS + shadcn/ui	Rapid UI development, consistent design system, accessible components
State Management	Zustand + TanStack Query	Lightweight global state + powerful server state caching and sync
Backend Framework	Node.js + Express (or Fastify)	Fast, lightweight, TypeScript-native, massive ecosystem
API Style	REST + WebSocket (Socket.IO)	REST for CRUD, WebSocket for real-time (tracking, chat, dispatch)
Database	PostgreSQL (Supabase or self-hosted)	Relational integrity for users, orders, finances; row-level security
ORM	Prisma	Type-safe database access, migrations, schema management
Cache	Redis	Session store, real-time data, job queues, leaderboard caching
Auth	NextAuth.js (Auth.js) + custom JWT	Social login, phone OTP, session management, role-based access
File Storage	Cloudflare R2 or AWS S3	Documents, photos, training videos, certificates
Real-time	Socket.IO	Live tracking, chat, dispatch updates, notifications
Maps	Mapbox GL JS (or Google Maps)	Turn-by-turn nav, geocoding, heatmaps, geofencing
Payments	Paystack + Flutterwave (Africa), Stripe (global)	Mobile money, card payments, instant payouts
Push Notifications	Web Push API + Firebase Cloud Messaging	PWA push notifications across devices
Email	SendGrid or Resend	Transactional and marketing emails
SMS	Twilio or Africa's Talking	OTP, alerts, fallback notifications
Search	Meilisearch	Fast search for riders, orders, training content
Background Jobs	BullMQ (Redis-backed)	Payout processing, notifications, reports, AI tasks
Monitoring	Sentry + Uptime monitoring	Error tracking, performance monitoring
CI/CD	GitHub Actions	Automated testing, build, deploy pipeline
Hosting	Vercel (frontends) + Railway/Render (backend)	Serverless frontend, managed backend, easy scaling
CDN	Cloudflare	Global edge caching for PWA assets

Project Structure (Monorepo)

```
riderguy/
apps/
  rider/              # Rider PWA (Next.js)
  client/             # Client PWA (Next.js)
  admin/              # Admin Portal (Next.js)
  marketing/          # Marketing Website (Next.js)
  api/                # Backend API (Node.js + Express)
packages/
  ui/                 # Shared UI component library (shadcn/ui based)
  types/              # Shared TypeScript types & interfaces
  utils/              # Shared utility functions
  config/             # Shared configuration (tailwind, eslint, tsconfig)
  database/           # Prisma schema, migrations, seed scripts
  validators/         # Shared Zod validation schemas
docs/                  # Documentation
scripts/               # Build, deploy, and maintenance scripts
.github/               # CI/CD workflows
turbo.json             # Turborepo config
package.json           # Root package.json
README.md
```

Phase 1 — Foundation (MVP)

Goal: Get core delivery operations running end-to-end. A rider can sign up, get verified, accept a delivery, navigate, confirm delivery, and get paid. A client can request a delivery, track it, and pay. An admin can manage it all. A marketing site attracts sign-ups.

Duration: ~12–16 weeks (Sprints 1–8)

Sprint 1 — Project Bootstrap & Infrastructure (Week 1–2)

The skeleton of everything.

#	Task	Platform	Details
1.1	Monorepo setup	All	Initialize Turborepo, configure workspaces for all 5 apps + shared packages
1.2	TypeScript config	All	Base tsconfig, per-app configs, path aliases
1.3	Shared packages	All	Create `types`, `utils`, `config`, `validators`, `ui` packages
1.4	Tailwind + Design System	All	Configure Tailwind, set up color palette, typography, spacing tokens per brand guidelines
1.5	shadcn/ui setup	All	Install and configure component primitives (Button, Input, Card, Dialog, etc.)
1.6	Database schema (v1)	Backend	Prisma schema: Users, Riders, Clients, Admins, Vehicles, Documents, Zones
1.7	Database setup	Backend	PostgreSQL instance (Supabase), run initial migration, seed dev data
1.8	API scaffold	Backend	Express/Fastify app, folder structure (routes, controllers, services, middleware)
1.9	Auth system (v1)	Backend	Phone OTP + email/password registration, JWT tokens, refresh tokens, role-based middleware
1.10	CI/CD pipeline	Infra	GitHub Actions: lint, type-check, test, build on every PR
1.11	Environment config	Infra	.env management, per-environment configs (dev, staging, prod)
1.12	PWA manifests	Rider, Client	Web app manifests, service worker shells, install prompts
1.13	Error handling	Backend	Global error handler, error codes, structured error responses
1.14	Logging setup	Backend	Structured logging (pino or winston), request ID tracing

Deliverable: All apps boot locally. Database runs. Auth works. CI passes. PWA manifests installed.

Sprint 2 — Auth, Onboarding & User Identity (Week 3–4)

Every user type can register and authenticate.

#	Task	Platform	Details
2.1	Registration API	Backend	Endpoints: register (rider, client, partner, admin), verify OTP, resend OTP
2.2	Login API	Backend	Endpoints: login (phone/email), token refresh, logout, session list
2.3	Profile API	Backend	Endpoints: get/update profile, upload avatar, change password
2.4	Role system	Backend	User roles enum (rider, client, business_client, partner, dispatcher, admin, super_admin), middleware guards
2.5	Rider sign-up flow	Rider PWA	Multi-step form: personal info → phone verification → zone selection → referral code
2.6	Client sign-up flow	Client PWA	Simple form: name, phone, email → OTP verification → done
2.7	Admin login page	Admin Portal	Email/password login with MFA support (phase 2)
2.8	Marketing site shell	Marketing	Next.js app with layout, header, footer, homepage hero section
2.9	Shared auth context	All frontends	Auth provider component: login state, token storage, auto-refresh, protected routes
2.10	Session management	Backend	Active sessions list, token revocation, device tracking

Deliverable: Riders, clients, and admins can sign up and log in. Tokens work. Protected routes enforced.

Sprint 3 — Rider Onboarding & Document Verification (Week 5–6)

Riders submit their documents and go through activation.

#	Task	Platform	Details
3.1	Document upload API	Backend	Endpoints: upload document (ID, license, insurance, vehicle reg), get docs, update status
3.2	File storage integration	Backend	S3/R2 upload with signed URLs, file type validation, size limits, virus scanning
3.3	Vehicle registration API	Backend	Endpoints: register vehicle (type, make, model, year, plate, photos), update, remove
3.4	Onboarding status API	Backend	Endpoint: get onboarding progress (checklist of completed/pending steps)
3.5	Document upload screens	Rider PWA	Step-by-step document upload: ID → license → insurance → vehicle photos, with camera integration
3.6	Selfie capture	Rider PWA	Selfie screen with liveness guidance (face in frame, good lighting)
3.7	Vehicle registration screen	Rider PWA	Form: vehicle type, details, condition photos upload
3.8	Onboarding checklist UI	Rider PWA	Visual progress tracker showing all steps, completion status, what's next
3.9	Application review queue	Admin Portal	List of pending rider applications, click to review documents, approve/reject/request-more
3.10	Document viewer	Admin Portal	Full-screen document viewer with zoom, rotate, approve/reject per document
3.11	Admin notification	Backend	Notify admin when new application submitted; notify rider when status changes
3.12	Zone management (basic)	Backend + Admin	CRUD for zones (name, polygon coordinates, status); assign riders to zones

Deliverable: Full rider onboarding pipeline — sign up, upload docs, get reviewed, get activated. Admins can review and approve.

Sprint 4 — Core Delivery Flow — Order Creation & Dispatch (Week 7–8)

A client can create a delivery. It gets dispatched to a rider.

#	Task	Platform	Details
4.1	Order schema	Backend	Prisma model: Orders (status, pickup, dropoff, package_type, price, rider_id, client_id, timestamps)
4.2	Pricing engine (v1)	Backend	Calculate price: base fare + per-km rate + package type multiplier. Config-driven.
4.3	Order creation API	Backend	Endpoints: estimate price, create order, get order details, list orders
4.4	Geocoding integration	Backend	Address → coordinates (Mapbox or Google Geocoding API), autocomplete endpoint
4.5	Distance calculation	Backend	Calculate distance + estimated duration between pickup and dropoff
4.6	Manual dispatch API	Backend	Endpoint: assign rider to order, reassign, unassign
4.7	Order status machine	Backend	Status flow: pending → assigned → pickup_en_route → picked_up → delivering → delivered / failed
4.8	Delivery request screen	Client PWA	Form: pickup address (autocomplete) → dropoff address → package type → special instructions → price estimate → confirm
4.9	Order confirmation screen	Client PWA	Order summary, estimated time, “finding a rider” state
4.10	Order list / history	Client PWA	List of all orders with status badges, tap to see details
4.11	Job feed (rider)	Rider PWA	Live list of available jobs in rider’s zone. Each card shows: pickup, dropoff, distance, pay, package type
4.12	Accept job flow	Rider PWA	Tap job → see details → accept → status changes to pickup_en_route
4.13	Dispatch dashboard (basic)	Admin Portal	List of all active orders, filter by status/zone. Manual assign rider to order.
4.14	Notification triggers	Backend	Push notification when: order created (to nearby riders), order assigned (to rider + client), status changes

Deliverable: End-to-end: client creates order → admin assigns rider (or rider accepts from feed) → order is in progress.

Sprint 5 — Delivery Execution & Proof of Delivery (Week 9–10)

Riders navigate, pick up, deliver, and confirm. Clients track in real time.

#	Task	Platform	Details
5.1	Real-time location API	Backend	WebSocket channel: rider emits GPS coords every 5–10 seconds while on active delivery
5.2	Location tracking service	Rider PWA	Background geolocation via service worker, battery-optimized, queues if offline
5.3	Static tracking API	Backend	REST endpoint: get latest rider location for a given order (for clients)
5.4	Navigation screen	Rider PWA	Mapbox map with route line, turn-by-turn directions, ETA, re-route button
5.5	Pickup confirmation	Rider PWA	At pickup: confirm arrival → photo capture or code entry → status to “picked_up”
5.6	In-transit screen	Rider PWA	Navigation to dropoff, status “delivering”, quick-message buttons to client
5.7	Proof of delivery	Rider PWA	At dropoff: photo capture / recipient signature (canvas) / PIN entry → complete delivery
5.8	Failed delivery flow	Rider PWA	Can’t deliver → call customer → wait timer → photo evidence → mark as failed with reason
5.9	Live tracking map	Client PWA	Real-time map showing rider’s position, route, ETA countdown
5.10	Status timeline	Client PWA	Vertical timeline showing every status change with timestamp
5.11	In-app messaging (v1)	Backend + Both	Simple text messaging between rider and client during active delivery (WebSocket)
5.12	Delivery completion handler	Backend	On delivery complete: update order status, calculate earnings, credit rider wallet, notify client
5.13	Rating & review API	Backend	Client rates rider (1-5 stars + optional comment) after delivery
5.14	Rating screen	Client PWA	Post-delivery: rate the rider, optional tip, optional feedback
5.15	Delivery summary	Rider PWA	After completion: earnings for this delivery, running daily total, next job prompt

Deliverable: Full delivery lifecycle working end-to-end with live tracking and proof of delivery.

Sprint 6 — Wallet, Payments & Earnings (Week 11–12)

Riders get paid. Clients pay. Money moves.

#	Task	Platform	Details
6.1	Wallet schema	Backend	Prisma models: Wallets, Transactions (type, amount, status, metadata, timestamps)
6.2	Wallet API	Backend	Endpoints: get balance, transaction history (paginated, filterable), wallet summary
6.3	Earnings credit	Backend	After delivery completion: auto-credit rider wallet (base pay + tips - commission)
6.4	Payment gateway integration	Backend	Paystack/Flutterwave: charge client card/mobile money at order creation
6.5	Client payment flow	Client PWA	Add payment method (card, mobile money), pay at checkout, payment confirmation
6.6	Withdrawal API	Backend	Endpoints: request withdrawal (to bank/mobile money), process withdrawal, withdrawal history
6.7	Payout processing	Backend	BullMQ job: batch-process pending withdrawals daily (or instant for eligible riders)
6.8	Rider wallet screen	Rider PWA	Balance card, recent transactions list, withdraw button, earnings chart (daily/weekly)
6.9	Transaction detail	Rider PWA	Tap any transaction: full details (order link, breakdown, timestamp)
6.10	Withdrawal screen	Rider PWA	Choose method (bank/mobile money) → enter amount → confirm → processing state
6.11	Admin payout dashboard	Admin Portal	List of pending/processed withdrawals, approve/hold/investigate, bulk payout trigger
6.12	Commission tracking	Backend + Admin	Platform commission calculated per delivery; visible in admin financial reports
6.13	Tip handling	Backend	Tips tracked separately, 100% credited to rider, visible in transaction history
6.14	Receipt generation	Backend	Auto-generate delivery receipt (PDF or email) for client after payment

Deliverable: Full payment cycle — client pays, rider earns, rider withdraws. Admin controls payouts.

Sprint 7 — Admin Portal Core & Marketing Site (Week 13–14)

Admin portal becomes operational. Marketing site goes live.

#	Task	Platform	Details
7.1	Admin dashboard home	Admin Portal	KPI cards: total riders, active riders, total deliveries (today/week/month), revenue, pending applications
7.2	Rider management	Admin Portal	Rider list with search/filter, rider profile view, edit status (active/suspended/deactivated), view documents
7.3	Client management	Admin Portal	Client list with search/filter, client profile view, order history, account status
7.4	Order management	Admin Portal	All orders list with filters (status, date, zone, rider, client), order detail view, manual status update
7.5	Zone management UI	Admin Portal	Map-based zone editor: draw polygons, name zones, set status, assign pricing
7.6	Pricing config UI	Admin Portal	Form to set: base fare, per-km rate, package multipliers, minimum fare, commission rate per zone
7.7	Basic analytics	Admin Portal	Charts: deliveries over time, revenue over time, rider sign-ups, active riders trend
7.8	Marketing homepage	Marketing	Full responsive homepage: hero, value props, how it works, social proof, rider CTA, client CTA
7.9	For Riders page	Marketing	Landing page: benefits, career path, community, sign-up CTA → links to rider app
7.10	For Businesses page	Marketing	Landing page: delivery network, reliability, pricing, sign-up CTA → links to client app
7.11	About page	Marketing	Mission, values, team, story
7.12	SEO foundations	Marketing	Meta tags, OG tags, structured data, sitemap.xml, robots.txt
7.13	Contact page	Marketing	Contact forms for: riders, businesses, partners, general inquiries
7.14	Legal pages	Marketing	Terms of service, privacy policy (template-based, legal review later)

Deliverable: Admin portal is functional for day-to-day operations. Marketing site is live and driving sign-ups.

Sprint 8 — MVP Polish, Testing & Launch Prep (Week 15–16)

Stabilize, test, and prepare for first real users.

#	Task	Platform	Details
8.1	End-to-end testing	All	Full user journey testing: rider onboarding → client order → delivery → payment → review
8.2	PWA optimization	Rider, Client	Service worker caching strategy, offline fallback pages, install prompt timing
8.3	Performance audit	All	Lighthouse audit, bundle size analysis, image optimization, lazy loading
8.4	Mobile responsiveness	All	Test on real devices (Android, iOS Safari), fix layout issues, touch targets
8.5	Error states	All	Empty states, loading skeletons, error boundaries, retry logic, offline indicators
8.6	Notification system (v1)	Backend	Push notifications via FCM for: new job, order status, approval status, payment received
8.7	Email templates	Backend	Transactional emails: welcome, verification, order confirmation, delivery receipt, payout confirmation
8.8	Rider go-online flow	Rider PWA	Toggle online/offline, zone confirmation, availability indicator
8.9	Security audit	Backend	Input validation, SQL injection prevention, rate limiting, CORS config, token security
8.10	Staging deployment	Infra	Deploy all apps to staging environment, end-to-end smoke tests
8.11	Production deployment	Infra	Deploy to production, DNS config, SSL certs, monitoring setup
8.12	Seed data	Backend	Realistic demo data for staging: riders, clients, orders, zones
8.13	Admin user setup	Backend	Create initial super_admin account, document admin access procedures
8.14	Launch checklist	Docs	Go-live checklist: DNS, monitoring, backups, error tracking, support channel

Deliverable: MVP LAUNCH — Rider app, client app, admin portal, and marketing site are live.

Phase 2 — Engagement & Intelligence

Goal: Make the platform sticky. Gamification, community, training, smart dispatch, partner program, business clients, and advanced financial tools. This is where RiderGuy stops being “just another delivery app” and becomes a career platform.

Duration: ~16–20 weeks (Sprints 9–18)

Sprint 9 — Gamification Engine: XP, Levels & Badges (Week 17–18)

#	Task	Platform	Details
9.1	XP schema	Backend	Prisma models: XP_Ledger (rider_id, action, points, metadata, timestamp), Rider_Level (current_xp, level)
9.2	XP calculation service	Backend	Award XP for: delivery complete, on-time, 5-star rating, training completion, referral, streak
9.3	Level thresholds	Backend	7-tier system with configurable XP thresholds, auto-level-up logic
9.4	Badge schema & service	Backend	Badge definitions, achievement rules engine, badge awarding on trigger events
9.5	XP events integration	Backend	Hook into delivery completion, rating received, training complete events to trigger XP awards
9.6	XP & level display	Rider PWA	Profile section: XP bar, current level, next level progress, XP history feed
9.7	Badge showcase	Rider PWA	Badge grid on profile, tap for details, “New badge earned!” celebration animation
9.8	Level-up notification	Rider PWA	Full-screen celebration when leveling up, showing new perks unlocked
9.9	Admin badge management	Admin Portal	CRUD for badge definitions, manual badge awarding, XP adjustment tools
9.10	Level perks enforcement	Backend	Commission rate adjustment per level, job access filtering by level

Sprint 10 — Leaderboards, Challenges & Rewards Store (Week 19–20)

#	Task	Platform	Details
10.1	Leaderboard API	Backend	Endpoints: zone/city/national/global leaderboards, by category, time period. Redis-cached.
10.2	Leaderboard screens	Rider PWA	Tabbed leaderboard: region, category, time range. Highlight rider's own position.
10.3	Streak tracking	Backend	Track consecutive active days, award streak badges, bonus XP for streaks
10.4	Challenges engine	Backend	Schema: Challenges (type, criteria, reward, start/end dates). Daily/weekly/monthly auto-creation.
10.5	Active challenges	Rider PWA	"Challenges" tab: active challenges with progress bars, rewards preview, time remaining
10.6	Rewards store schema	Backend	Store items, point costs, inventory, redemption tracking
10.7	Rewards store UI	Rider PWA	Browse rewards, filter by category/cost, redeem with points, redemption history
10.8	Admin challenge manager	Admin Portal	Create/edit/delete challenges, set criteria, rewards, target audience
10.9	Admin rewards manager	Admin Portal	Manage store items, inventory, pricing, featured items
10.10	Bonus XP events	Admin Portal	Create time-limited XP multiplier events (double XP weekend, etc.)

Sprint 11 — Rider Community: Chat & Forums (Week 21–22)

Sprint 12 — Community: Mentorship, Events & Rider Identity (Week 23–24)

#	Task	Platform	Details
12.1	Mentorship schema	Backend	Mentor-mentee pairings, status, check-in tracking, completion criteria
12.2	Mentor matching API	Backend	Auto-match Level 5+ riders with new riders in same zone; manual override
12.3	Mentorship dashboard	Rider PWA	For mentors: list of mentees, check-in reminders, progress. For mentees: mentor profile, chat, activity.
12.4	Events schema	Backend	Events (title, description, date, location/virtual_link, zone, type, capacity)
12.5	Events API + UI	Backend + Rider	Browse upcoming events, RSVP, calendar view, event reminders
12.6	Rider profile upgrade	Rider PWA	Full profile: photo, bio, vehicle, badges, certifications, stats, XP, level
12.7	Public rider card	Rider PWA	Shareable digital card with QR code, verified credentials, public profile URL
12.8	Rider-of-the-month	Backend + Admin	Automated selection criteria OR manual admin pick. Featured on community feed.
12.9	Feature request board	Rider PWA	Submit ideas, upvote others' ideas, status tracking (submitted, reviewed, planned, shipped)
12.10	Rider stories/spotlight	Rider PWA + Marketing	Featured rider profiles with stories, visible in app community hub and marketing site

Sprint 13 — Training Academy (LMS) (Week 25–26)

Sprint 14 — Smart Dispatch & Auto-Assignment (Week 27–28)

#	Task	Platform	Details
14.1	Dispatch scoring algorithm	Backend	Score riders for each order: distance weight + rating weight + load weight + level weight + vehicle match
14.2	Auto-dispatch service	Backend	On new order: calculate scores for available riders in zone, offer to top-scored rider, timeout → next
14.3	Dispatch queue	Backend	BullMQ job: manage offer lifecycle (offer → accept/decline/timeout → next rider or escalate)
14.4	Batch delivery logic	Backend	Group nearby pickups/dropoffs into multi-stop routes with optimized order
14.5	Surge pricing engine	Backend	Calculate supply/demand ratio per zone. Auto-apply multiplier. Configurable thresholds.
14.6	Dispatch dashboard upgrade	Admin Portal	Real-time map with rider positions, order markers, drag-drop assignment, auto-dispatch toggle, surge indicators
14.7	Rider availability API	Backend	Track online status, current load, last active, location freshness
14.8	Smart job feed	Rider PWA	Job feed sorted by relevance (proximity, pay, match score), not just time
14.9	Surge indicator	Rider PWA	“High demand in your zone” notification + surge multiplier shown on job cards
14.10	Dispatch analytics	Admin Portal	Match rate, average assignment time, decline rate, surge history

Sprint 15 — Partner Program & Referral System (Week 29–30)

#	Task	Platform	Details
15.1	Partner schema	Backend	Partners (user_id, referral_code, tier, status), Partner_Commissions, Partner_Riders (attribution)
15.2	Partner registration API	Backend	Register as partner, generate referral code/link, KYC for partners
15.3	Referral attribution	Backend	On rider sign-up: if referral code entered, create attribution record, track through activation & first delivery
15.4	Commission engine	Backend	Calculate and accrue commissions: sign-up bonus, activation bonus, ongoing % for X months
15.5	Partner portal shell	Partner (or Rider app sub-section)	Dashboard: total recruits, active riders, pending/earned commissions, referral link + QR
15.6	Recruited riders list	Partner portal	List all recruited riders with status (pending, active, inactive), delivery count, join date
15.7	Commission history	Partner portal	Transaction log of all commission accruals and payouts
15.8	Partner payout	Backend	Partner withdrawal to bank/mobile money, admin approval flow
15.9	Partner tier system	Backend	Auto-upgrade partner tier based on recruit count, adjust commission rates
15.10	Admin partner management	Admin Portal	Partner list, review applications, adjust tiers, view recruitment stats, process payouts
15.11	Marketing: For Partners page	Marketing	Landing page: how it works, commission tiers, sign-up CTA
15.12	Partner marketing materials	Partner portal	Downloadable flyers, social media assets, scripts

Sprint 16 — Business Client Dashboard (Week 31–32)

#	Task	Platform	Details
16.1	Business account schema	Backend	Business_Accounts (company_name, type, billing), Team_Members (role, permissions)
16.2	Business registration	Client PWA	Company sign-up flow: business details, admin user, billing setup
16.3	Team management	Client PWA	Invite team members, assign roles (admin, dispatcher, viewer), manage permissions
16.4	Bulk order upload	Client PWA + Backend	CSV upload: parse, validate, preview, create multiple orders at once
16.5	Recurring schedules	Backend	Cron-triggered: auto-create orders from saved schedule templates
16.6	Multi-branch	Client PWA	Add store locations, select pickup branch per order, cross-branch analytics
16.7	Business analytics	Client PWA	Order volume, cost per delivery, on-time rate, rider performance, monthly spend chart
16.8	Invoice generation	Backend + Admin	Monthly invoice with line items per delivery, downloadable PDF
16.9	API documentation	Docs	OpenAPI spec, getting started guide, auth flow, endpoint reference
16.10	Webhook system	Backend	Configurable webhooks: order.created, order.delivered, etc. Retry logic, signature verification.
16.11	API key management	Admin Portal	Issue API keys per business client, track usage, rate limiting, revoke

Sprint 17 — Advanced Wallet & Financial Features (Week 33–34)

#	Task	Platform	Details
17.1	Earnings analytics	Rider PWA	Charts: daily/weekly/monthly earnings, comparisons, per-delivery average, peak hours heatmap
17.2	Expense tracker	Rider PWA	Log fuel, maintenance costs. Calculate net earnings. Monthly summary.
17.3	Savings goals	Rider PWA + Backend	Create savings goals (target, auto-deduction %), track progress, withdraw from savings
17.4	Scheduled withdrawals	Backend	Auto-withdraw on schedule (daily/weekly), configurable per rider
17.5	Tip management	Rider PWA	Tips breakdown, tip trends, “total tips this month” card
17.6	Financial insights	Rider PWA	AI-generated insights: “You earned more on Tuesdays”, “Your busiest zone is X”
17.7	Earnings breakdown	Rider PWA	Per delivery: base pay, distance bonus, surge bonus, tip, commission deduction → net
17.8	Admin financial reports	Admin Portal	Revenue dashboards, payout reports, commission analytics, margin tracking
17.9	Financial reconciliation	Admin Portal	Match credits vs debits, flag discrepancies, resolution workflow
17.10	Tax summary	Rider PWA + Backend	Downloadable annual/quarterly earnings summary for tax purposes

Sprint 18 — Push Notifications, Comms & Phase 2 Polish (Week 35–36)

#	Task	Platform	Details
18.1	Transactional notifications	Backend	Push notifications for: delivery assigned, order ready, payment received, payout processed
18.2	Earnings milestone notifications	All apps	Notify when rider hits milestones (e.g. “You earned 500 today”)
18.3	Inactivity nudges	All apps	Nudge inactive riders to come back online (e.g. “Peak hours starting soon”)
18.4	Peak hours alerts	Backend + Admin	Notify riders when peak hours approaching, based on historical data
18.5	SMS fallbacks	Backend	Critical notifications via SMS when push fails
18.6	Admin broadcast messages	Admin Portal	Admin can send bulk messages to riders (e.g. “Holiday schedule”)
18.7	Support chat	Backend + UI	Rider-to-support chat in app, admin-side chat interface
18.8	Onboarding checklist UI	Rider PWA	Interactive checklist on first login: profile, bank, vehicle, first delivery
18.9	Empty states & error UX	UI	Polish empty states, error messages, loading states
18.10	Phase 2 bugfixes	All	Fixes from Phase 2 testing

Deliverable: PHASE 2 COMPLETE — Gamification, community, training, smart dispatch, partner program, business dashboard, advanced finance.

Phase 3 — Scale & Optimize

Goal: AI intelligence, multi-region expansion, fraud prevention, e-commerce integrations, welfare programs, and white-label capabilities.

Duration: ~16–20 weeks (Sprints 19–28)

Sprint 19–20 — AI & Predictive Analytics (Week 37–40)

#	Task	Details
19.1	Demand forecasting model	Historical data analysis → predict delivery volume by zone/hour. Display in admin dashboard.
19.2	Dynamic pricing engine	Auto-adjust pricing based on real-time supply/demand ratio. Configurable bounds and rules.
19.3	Churn prediction	ML model: identify riders/clients likely to churn based on activity patterns. Alert admin.
19.4	Earnings optimization	AI suggestions for riders: “Move to Zone B for 30% more orders this hour”
19.5	Route learning	Learn from rider route choices, improve navigation suggestions over time
19.6	Smart notifications	Optimize send timing based on user engagement patterns per time-of-day
19.7	AI chatbot (v1)	NLP-powered support bot for common rider/client queries. Escalation to human when needed.
19.8	Predictive admin dashboard	Forecasting widgets: expected orders tomorrow, suggested rider staffing, risk alerts

Sprint 21–22 — Fraud Detection & Safety Systems (Week 41–44)

#	Task	Details
21.1	Fraud detection	Rules: multi-account, GPS spoofing, fake deliveries, suspicious patterns. Flag for review.
21.2	Behavioral analytics	Track anomalies in rider behavior. Alert on deviations from baseline.
21.3	Photo verification	Random selfie checks during delivery. Compare to profile photo.
21.4	Trust scores	Calculate trust score for riders and clients. Affects visibility and priority.
21.5	Escalation	Escalation queue for flagged cases. Admin review workflow.
21.6	Dispute resolution	Structured dispute flow: rider vs. client. Evidence upload. Admin mediation.
21.7	Automated actions	Auto-suspend for critical violations. Warnings for minor. Appeal process.
21.8	Legal compliance	Terms of service, liability boilerplate, insurance requirements.
21.9	Insurance partnership	Partnership with insurance provider. Coverage for riders. Claims process.
21.10	Incident reporting	Rider reports accidents, theft, harassment. Admin tracks resolution.
21.11	Fraud dashboard	Admin view: flagged accounts, pending reviews, fraud trends, resolution stats.

Sprint 23–24 — Multi-Region, Localization & Compliance (Week 45–48)

#	Task	Details
23.1	Multi-region architecture	Region as top-level entity: separate zones, pricing, policies, staff per region
23.2	Localization framework	i18n setup: all user-facing strings extracted, translation file per language
23.3	Multi-currency	Currency per region, proper formatting, FX for cross-border if applicable
23.4	Regional compliance	Configurable document requirements, labor law compliance, regulatory reporting per region
23.5	Regional admin access	Admin roles scoped to regions: regional managers see only their region
23.6	Data residency	Database-per-region or row-level region isolation for data privacy compliance
23.7	GDPR/Privacy tools	Data export, account deletion, consent management, cookie compliance
23.8	Regulatory reporting	Auto-generate reports required by local transport/labor authorities per region

Sprint 25–26 — E-commerce Integration & White-Label (Week 49–52)

#	Task	Details
25.1	Public REST API (v2)	Full-featured API with versioning, rate limiting, pagination, filtering
25.2	Shopify integration	Shopify app: auto-create RiderGuy delivery when Shopify order placed
25.3	WooCommerce plugin	WordPress plugin for WooCommerce → RiderGuy delivery automation
25.4	POS integration framework	Standard integration layer for restaurant/retail POS systems
25.5	White-label config	Custom branding (logo, colors, domain) per business client for tracking pages
25.6	Branded tracking pages	Client-branded delivery tracking URLs for end customers
25.7	Returns management	Initiate return pickups, reverse logistics flow, return-to-sender tracking
25.8	Cash-on-delivery	COD collection tracking, reconciliation, daily settlement reports
25.9	SDK / embeddable widget	JS widget businesses can embed on their site for delivery requests

Sprint 27–28 — Rider Welfare & Benefits Programs (Week 53–56)

#	Task	Details
27.1	Insurance marketplace	In-app enrollment for health, accident, vehicle insurance via partner providers
27.2	Microloan system	Loan application, approval engine (based on earnings history), auto-repayment
27.3	Emergency fund	Rider contributions, application process, admin review, payouts
27.4	Vehicle financing	Partner integration: motorcycle financing with repayment via earnings deduction
27.5	Gear subsidies & marketplace	Discounted gear ordering, verified seller marketplace for equipment
27.6	Rest stop network	Map of partner locations with rider discounts, directions, amenities info
27.7	Vehicle maintenance network	Verified mechanics directory, discounted service bookings, maintenance reminders
27.8	Mental health resources	Content hub: stress management, counseling service directory
27.9	Financial literacy	In-app budgeting tools, educational content, savings tips
27.10	Rain-day guarantee	Config: minimum earnings guarantee during extreme weather events

Deliverable: PHASE 3 COMPLETE — AI intelligence, multi-region, fraud prevention, e-commerce integrations, welfare ecosystem.

Phase 4 — Ecosystem & Dominance

Goal: Transform RiderGuy from a product into, an ecosystem. API-driven platform marketplace, fintech services, academy licensing, data products, and global expansion infrastructure.

Duration: ~20+ weeks (Sprints 29–40+)

Sprint 29–30 — API Marketplace & Developer Ecosystem

29.1	Public API gateway	Rate limiting, auth, versioning, documentation, developer portal
29.2	API key infrastructure	Key issuance, developer onboarding, usage tracking
29.3	Webhook for third parties	Event notifications, retry logic, delivery logs
29.4	Partner dashboards	Usage analytics, revenue reports, integration status
29.5	Sandbox environment	Test environment, mock data, documentation

Sprint 31–32 — RiderGuy Financial Services

#	Task	Details
31.1	Savings accounts	Interest-bearing savings within the wallet
31.2	Peer-to-peer transfers	Send money to other riders, split payments
31.3	Investment products	Micro-investment options for riders (partner with fintech)
31.4	Insurance brokering	Full insurance comparison and purchasing within the app
31.5	Credit scoring	Build rider credit profiles based on platform activity and earnings history

Sprint 33–34 — Academy as a Product

#	Task	Details
33.1	Standalone academy	Academy accessible outside RiderGuy for other delivery companies
33.2	White-label LMS	License the training platform to partners with custom branding
33.3	Content marketplace	Training content creators can publish and sell courses
33.4	Industry certifications	Partner with transport authorities for recognized rider certifications
33.5	Live training infrastructure	Scalable live session system for instructor-led training at scale

Sprint 35–36 — Data & Insights Products

#	Task	Details
35.1	Logistics intelligence	Anonymized delivery data sold to businesses and city planners
35.2	Supply/demand insights	Real-time and historical supply/demand data for business planning
35.3	City-level reports	Traffic patterns, delivery density, coverage maps for municipal use
35.4	Custom analytics	Bespoke data projects for enterprise clients

Sprint 37–38 — Social Impact & Strategic Partnerships

#	Task	Details
37.1	Youth employment program	Structured onboarding for unemployed youth, subsidized first month
37.2	Women in delivery	Targeted program for women riders: safety features, community, incentives
37.3	Rural access initiative	Expand deliveries to underserved areas with adjusted pricing/economics
37.4	Vehicle manufacturer partnerships	Discounted bikes/scooters for riders through manufacturer deals
37.5	Telco partnerships	Discounted data plans for RiderGuy riders
37.6	Government partnerships	Regulatory collaboration, employment classification compliance
37.7	Rider cooperatives	Pilot rider-owned chapters with revenue sharing models

Sprint 39–40 — Global Scale & Infrastructure

#	Task	Details
39.1	Cross-border delivery	Multi-country delivery routing, customs integration where needed
39.2	Multi-cloud deployment	Deploy across multiple cloud regions for latency and compliance
39.3	Advanced observability	Distributed tracing, custom metrics, SLA monitoring, chaos engineering
39.4	Autonomous delivery prep	API abstractions for future drone/robot delivery integration
39.5	IPO-readiness	Governance systems, SOX compliance, board reporting dashboards
39.6	Enterprise security	SOC 2 compliance, penetration testing cadence, bug bounty program

Sprint Summary View

Sprint	Focus	Phase	Weeks
Sprint 1	Project Bootstrap & Infrastructure	Phase 1	1–2
Sprint 2	Auth, Onboarding & User Identity	Phase 1	3–4
Sprint 3	Rider Onboarding & Document Verification	Phase 1	5–6
Sprint 4	Core Delivery Flow — Order Creation & Dispatch	Phase 1	7–8
Sprint 5	Delivery Execution & Proof of Delivery	Phase 1	9–10
Sprint 6	Wallet, Payments & Earnings	Phase 1	11–12
Sprint 7	Admin Portal Core & Marketing Site	Phase 1	13–14
Sprint 8	MVP Polish, Testing & Launch Prep	Phase 1	15–16
Sprint 9	Gamification: XP, Levels & Badges	Phase 2	17–18
Sprint 10	Leaderboards, Challenges & Rewards Store	Phase 2	19–20
Sprint 11	Rider Community: Chat & Forums	Phase 2	21–22
Sprint 12	Mentorship, Events & Rider Identity	Phase 2	23–24
Sprint 13	Training Academy (LMS)	Phase 2	25–26
Sprint 14	Smart Dispatch & Auto-Assignment	Phase 2	27–28
Sprint 15	Partner Program & Referral System	Phase 2	29–30
Sprint 16	Business Client Dashboard	Phase 2	31–32
Sprint 17	Advanced Wallet & Financial Features	Phase 2	33–34
Sprint 18	Push Notifications, Comms & Phase 2 Polish	Phase 2	35–36
Sprint 19–20	AI & Predictive Analytics	Phase 3	37–40
Sprint 21–22	Fraud Detection & Safety Systems	Phase 3	41–44
Sprint 23–24	Multi-Region, Localization & Compliance	Phase 3	45–48
Sprint 25–26	E-commerce Integration & White-Label	Phase 3	49–52
Sprint 27–28	Rider Welfare & Benefits Programs	Phase 3	53–56
Sprint 29–30	API Marketplace & Developer Ecosystem	Phase 4	57–60
Sprint 31–32	RiderGuy Financial Services	Phase 4	61–64
Sprint 33–34	Academy as a Product	Phase 4	65–68
Sprint 35–36	Data & Insights Products	Phase 4	69–72
Sprint 37–38	Social Impact & Strategic Partnerships	Phase 4	73–76
Sprint 39–40	Global Scale & Infrastructure	Phase 4	77–80

Key Milestones

Milestone	Target	What It Means
M1: First Boot	Week 2	All apps run locally, database is live, auth works
M2: First Delivery	Week 10	A real delivery flows end-to-end: client orders → rider delivers → client receives
M3: First Payment	Week 12	Money flows: client pays → rider earns → rider withdraws
M4: MVP Launch	Week 16	All 4 platforms live. Riders can work. Clients can order. Admins manage. Marketing drives sign-ups.
M5: Engagement Loop	Week 24	Gamification, community, and mentorship create stickiness — riders want to stay
M6: Academy Live	Week 26	Riders can take courses, earn certs, and grow their careers on-platform
M7: Smart Dispatch	Week 28	AI-powered dispatch replaces manual — operations scale without linear staff growth
M8: Partner Engine	Week 30	Partners actively recruiting and earning — organic growth engine operational
M9: Business API	Week 32	Businesses integrate delivery into their own platforms via API
M10: Full Phase 2	Week 36	Platform is feature-rich, sticky, and differentiated — ready for aggressive growth
M11: AI-Powered	Week 40	Demand forecasting, dynamic pricing, churn prediction — platform thinks ahead
M12: Multi-Region	Week 48	Platform operates in multiple regions with localized everything
M13: Ecosystem	Week 56	Welfare, e-commerce integrations, white-label — platform is an ecosystem, not just an app
M14: Platform Company	Week 72+	API marketplace, financial services, academy product, data products — RiderGuy is a platform company

What to Build First (If Time Is Short)

If you need to get *something* live fast, here's the absolute minimum viable path:

1. **Sprint 1** — Monorepo, database, auth
2. **Sprint 2** — Rider sign-up, client sign-up
3. **Sprint 4** — Order creation, job feed, manual dispatch
4. **Sprint 5** — Live tracking, proof of delivery
5. **Sprint 6** — Wallet and payments

That's a working delivery platform in **~12 weeks.** Everything else layers on top.

RiderGuy Build Plan — From zero to the world's most supported rider network.